

Prometheus & PromQL

The Complete Concept-by-Concept Study Guide

Observability · Prometheus · PromQL · Instrumentation · Service Discovery · Alerting

Step-by-step · Diagrams · Code Examples · Purpose of Every Concept

Table of Contents

Part 1 — Observability Fundamentals

- 1.1 What is Observability?
- 1.2 The Three Pillars: Logs, Metrics, Traces
- 1.3 SLI, SLO and SLA

Part 2 — Prometheus Fundamentals

- 2.1 What is Prometheus?
- 2.2 Prometheus Architecture
- 2.3 Installing Prometheus
- 2.4 Node Exporter
- 2.5 Prometheus Configuration (prometheus.yml)
- 2.6 Authentication & TLS Encryption
- 2.7 The Prometheus Data Model (Metrics)
- 2.8 The Expression Browser
- 2.9 Running Prometheus in Docker
- 2.10 Monitoring Containers (cAdvisor)
- 2.11 Real-World Prometheus Use Cases
- 2.12 promtool

Part 3 — PromQL (Prometheus Query Language)

- 3.1 Introduction to PromQL & Data Types
- 3.2 Selectors and Label Matchers
- 3.3 Modifiers: offset and @
- 3.4 PromQL Practice Demo
- 3.5 Operators (Arithmetic, Comparison, Logical)
- 3.6 Vector Matching (on, ignoring, group_left/right)
- 3.7 Aggregation Operators (by, without)
- 3.8 PromQL Functions (rate, irate, etc.)
- 3.9 Subqueries
- 3.10 Histograms & Summaries + histogram_quantile()
- 3.11 Recording Rules
- 3.12 The HTTP API

Part 4 — Dashboarding & Visualization

- 4.1 Visualization Options Overview
- 4.2 Console Templates

Part 5 — Application Instrumentation

- 5.1 Client Libraries: Introduction
- 5.2 Instrumentation Basics (Flask + Counter)
- 5.3 Labels in Instrumentation
- 5.4 Histogram & Summary Instrumentation
- 5.5 The Gauge Metric Type
- 5.6 Naming & Instrumentation Best Practices

Part 6 — Service Discovery

- 6.1 Why Service Discovery?
- 6.2 File-based Service Discovery
- 6.3 AWS EC2 Service Discovery
- 6.4 Relabeling

Part 7 — Alerting

- 7.1 Alerting Overview & Alert Lifecycle
- 7.2 Labels vs Annotations
- 7.3 Alertmanager Architecture
- 7.4 Installing Alertmanager
- 7.5 Alertmanager Configuration (routing tree)
- 7.6 Receivers & Notifiers
- 7.7 End-to-End Alertmanager Demo
- 7.8 Silences

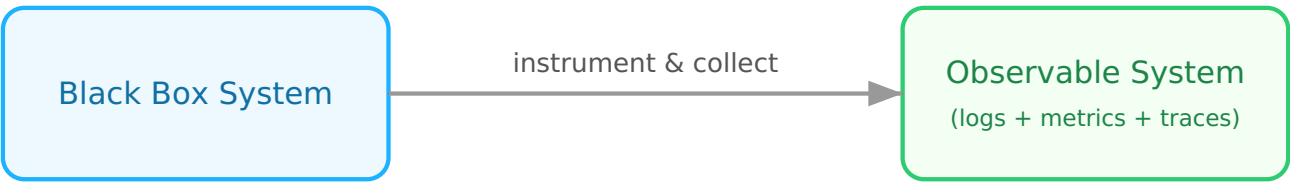
Part 1 — Observability Fundamentals

1.1 What is Observability?

Observability is the ability to infer the internal state of a system purely from the data it emits (logs, metrics, traces). It turns a system from a "black box" into something you can reason about when things go wrong.

Why it matters

As systems move from a single monolith to dozens of microservices, a failure can no longer be diagnosed by looking at one log file on one server. Observability gives engineers the flexibility to ask *new, unplanned* questions when an incident happens — not just the questions a pre-built dashboard anticipated.



Step-by-step: how observability helps troubleshooting

1. An alert fires: error rates suddenly rise.
2. You check **metrics** to see *which* service/endpoint is affected and *when* it started.
3. You check **traces** to see *where* in the request path (which service hop) latency or errors are concentrated.
4. You check **logs** for that exact service/timeframe to see the *detailed error message* or stack trace.
5. You now have root cause + context, instead of just "something is wrong."

1.2 The Three Pillars: Logs, Metrics, Traces

Pillar	What it gives you	Typical examples	Best for
Logs	Timestamped, detailed event records	Application logs, kernel logs, structured JSON logs	Forensic, "what exactly happened" analysis
Metrics	Aggregated numeric time series	CPU usage, request latency, error counts	Trend analysis, dashboards, alerting
Traces	End-to-end path of a single request across services	Trace IDs and spans (gateway → auth → user service)	Finding which hop in a distributed call added latency

Logs — example

```
Oct 26 19:35:00 ub1 kernel: e1000: enp0s3 NIC Link is Down
Oct 26 19:35:00 ub1 kernel: e1000 0000:00:03:0 enp0s3: Reset adapter
Oct 26 19:35:03 ub1 kernel: e1000: enp0s3 NIC Link is Up 1000 Mbps Full Duplex
```

Traces — concept diagram



One trace = the whole path; each hop is a "span" with its own duration

Metrics — example

```
node_filesystem_avail_bytes{fstype="vfat", mountpoint="/home"} 5000
```

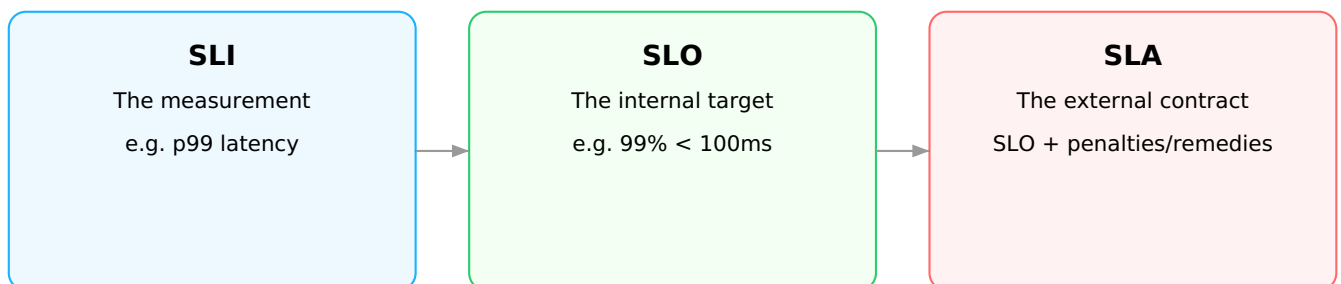
A metric has 4 parts: **name** (what), **labels** (dimensions to filter/group by), **value** (the number), **timestamp** (when it was recorded).

Where does Prometheus fit?

Prometheus is a **metrics-first** monitoring system. It scrapes numeric time series, evaluates rules, stores metrics, and triggers alerts. It is *not* a logging or tracing system — for full observability you pair it with a logging stack (ELK/EFK) and a tracing stack (Jaeger, Zipkin, OpenTelemetry).

1.3 SLI, SLO and SLA

These three concepts let a team turn "reliability" into something measurable and agreeable, rather than a vague feeling.



Service Level Indicator (SLI)

A quantitative metric measuring one aspect of user experience: request latency, error rate, throughput, saturation, availability. Good SLIs are **user-centric** — prefer "what the customer feels" over raw infrastructure metrics like CPU%.

Service Level Objective (SLO)

The target value/range for an SLI over a time window, e.g. "99% of requests < 100ms over a rolling 30-day window" or "99.9% availability over 30 days."

Common mistake

Never set an SLO to 100% — it's expensive, fragile, and leaves zero error budget for planned risk (deploys, experiments).

The gap between 100% and your SLO is your **error budget** — the amount of "acceptable failure" you can spend on releases, migrations, and risk before you must pause and focus purely on reliability.

Service Level Agreement (SLA)

A formal (often legal) contract that guarantees an SLO to a customer, with defined remedies (service credits, termination

rights) if it's missed.

Concept	Purpose	Example
SLI	Measurable indicator of behavior	request latency (p99) = 120ms
SLO	Internal target for an SLI	99% of requests < 100ms over 30 days
SLA	Contractual guarantee with remedies	99.9% uptime over 30 days → credits if violated

Why this matters for Prometheus users

You will use PromQL (Part 3) and specifically `histogram_quantile()` to actually *compute* your SLIs from raw metrics, and Alerting (Part 7) to notify the team when the error budget is burning too fast.

Part 2 — Prometheus Fundamentals

2.1 What is Prometheus?

Prometheus is an open-source monitoring and alerting toolkit. It:

- Collects numeric time-series metrics from configured **targets** (pull model).
- Stores them in a local **TSDB** (time-series database).
- Provides **PromQL** to query/aggregate that data.
- Integrates with **Alertmanager** to route notifications.

Purpose

Prometheus exists to answer "what is the current/historical numeric state of my systems, and should someone be paged about it?" It deliberately does NOT try to be a log store or a tracing system — it does one thing (metrics) very well.

How scraping works, step by step

1. You list targets in `prometheus.yml` (or discover them dynamically — Part 6).
2. Every `scrape_interval` (e.g. 15s), Prometheus sends an HTTP GET to each target's `/metrics` path.
3. The target responds with plain text in the Prometheus exposition format.
4. Prometheus parses each line into a metric name + labels + value, timestamps it with the scrape time, and appends it to the TSDB.
5. You (or an alert rule) query the TSDB using PromQL.

Example: a scrape config and the exposition format it consumes

```
# prometheus.yml
global:
  scrape_interval: 15s
scrape_configs:
  - job_name: 'node'
    static_configs:
      - targets: ['localhost:9100']
  - job_name: 'my-app'
    static_configs:
      - targets: ['my-app:8080']
```

```
# What the target's /metrics endpoint returns:
# HELP http_requests_total Total number of HTTP requests
# TYPE http_requests_total counter
http_requests_total{method="GET",handler="/api"} 1027
```

The 4 metric types (used everywhere in this guide)

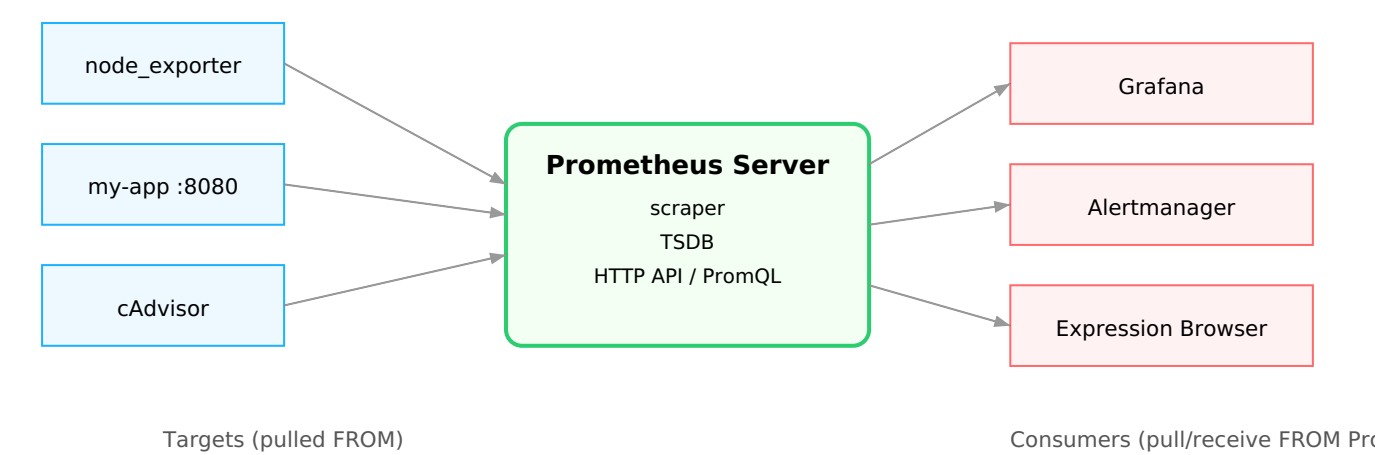
Type	Purpose	Use case	Example
Counter	Only increases (or resets on restart)	Total requests, errors	<code>http_requests_total</code>
Gauge	Goes up and down	Memory usage, in-progress requests	<code>process_resident_memory_bytes</code>
Histogram	Bucketed observation counts + sum/count	Latency distributions	<code>http_request_duration_seconds_bucket</code>
Summary	Client-computed quantiles + sum/count	Per-instance percentiles	<code>http_request_duration_seconds{quantile="0.5"}</code>

Prometheus is **not** a fit for events, raw system logs, or traces — only numeric time series.

2.2 Prometheus Architecture

The Prometheus *server* itself is three internal components working together:

Component	Responsibility
Data retrieval worker (scraper)	Polls each target's <code>/metrics</code> endpoint on schedule
TSDB	Stores scraped samples, optimized for high write volume + compression
HTTP server / query layer	Exposes the HTTP API + Web UI for PromQL queries



Exporters

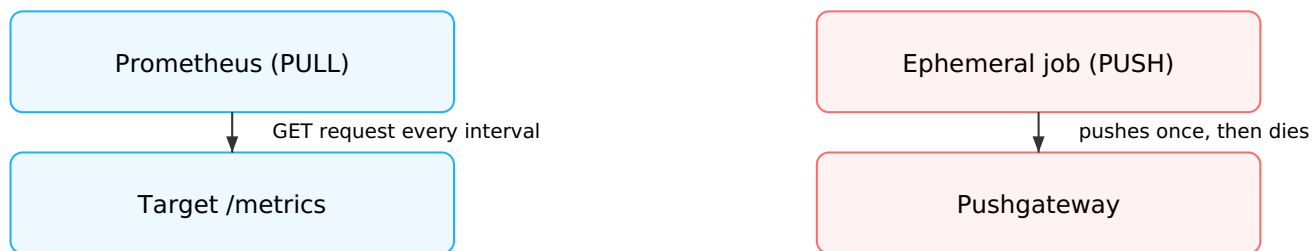
Most systems don't natively expose Prometheus-format metrics. An **exporter** is a small process that collects internal metrics from a system and republishes them at `/metrics` in the correct format.

Exporter	Use case
<code>node_exporter</code>	Linux host metrics (CPU, memory, disk, network)
<code>windows_exporter</code>	Windows server metrics
<code>mysqld_exporter</code>	MySQL/MariaDB metrics
<code>apache_exporter</code>	Apache HTTP Server
<code>haproxy_exporter</code>	HAProxy

Client libraries

For metrics that only *your own application code* knows (business logic, custom counters), you embed a client library (Go, Java, Python, Ruby, Rust, etc.) directly in the app — covered in depth in Part 5.

Pull vs Push



	Pull (Prometheus default)	Push
Who initiates	Prometheus server, on a schedule	The target itself sends data
Detecting "down"	Easy — a failed scrape = target down	Hard — silence could mean "healthy and idle" or "dead"
Best for	Long-running services	Short-lived/batch jobs (via Pushgateway)

Pushgateway is NOT a general push replacement

Only use it to buffer metrics from short-lived batch jobs that would otherwise disappear before a scrape could happen. Don't use it for long-lived services — instrument/expose those directly.

Service discovery (preview)

Static target lists don't scale in dynamic environments (Kubernetes, autoscaling). Prometheus supports Kubernetes, EC2, Consul, Azure, GCE, DNS, and file-based discovery to automatically keep its target list current. Covered fully in Part 6.

2.3 Installing Prometheus

1. **Download** the correct binary tarball from <https://prometheus.io/download/> :

```
wget https://github.com/prometheus/prometheus/releases/download/v2.37.0/prometheus-2.37.0.linux-amd64.tar.gz
```

2. **Extract** it:

```
tar xvf prometheus-2.37.0.linux-amd64.tar.gz
cd prometheus-2.37.0.linux-amd64/
ls -l
# prometheus      -> the server binary
# prometheus.yml   -> config file
# promtool         -> CLI validator
# consoles/, console_libraries/ -> UI templates
```

3. **Start** it:

```
./prometheus
```

You should see log lines like `msg="TSDB started"` and `msg="Server is ready to receive web requests."`

4. **Verify** via the Web UI at <http://localhost:9090> and run the query:

```
up
```

Result: `up{instance="localhost:9090", job="prometheus"}` with value `1` = target is UP; `0` = DOWN.

Production notes

Open port 9090 in the firewall if remote. Never run Prometheus as root in production — use a dedicated system user.

2.4 Node Exporter

The standard exporter for Linux host-level metrics (CPU, memory, disk, network, kernel stats). Listens on port **9100** by default.

1. Download & extract:

```
wget https://github.com/prometheus/node_exporter/releases/download/v1.3.1/node_exporter-1.3.1.linux-amd64.tar.gz
tar xvf node_exporter-1.3.1.linux-amd64.tar.gz
cd node_exporter-1.3.1.linux-amd64
./node_exporter
```

2. Verify locally:

```
curl localhost:9100/metrics
# TYPE node_uname_info gauge
node_uname_info{machine="x86_64",sysname="Linux",...} 1
```

3. Add it as a Prometheus scrape target:

```
scrape_configs:
  - job_name: 'node_exporter'
    static_configs:
      - targets: ['my-host.example.com:9100']
```

4. Run continuously via a **systemd** unit so it survives reboots (best practice).

Security

An exposed `/metrics` endpoint can leak system details. Restrict with a firewall, VPN, or the TLS/basic-auth setup in section 2.6.

2.5 Prometheus Configuration (prometheus.yml)

Prometheus is entirely YAML-driven. Two top-level sections matter most: `global` (defaults for every job) and `scrape_configs` (the actual jobs/targets).

```
global:
  scrape_interval: 15s      # how often to scrape targets
  evaluation_interval: 15s  # how often to evaluate alert/recording rules

alerting:
  alertmanagers:
    - static_configs:
        - targets: ["alertmanager:9093"]

rule_files:
  - "rules.yml"

scrape_configs:
  - job_name: "prometheus"
    static_configs:
      - targets: ["localhost:9090"]

  - job_name: "node"
    scrape_interval: 15s      # overrides global for THIS job only
    scrape_timeout: 5s
    scheme: https             # http by default
    metrics_path: /stats/metrics # defaults to /metrics
    static_configs:
      - targets: ["192.168.1.168:9100"]
```

Field	Purpose
<code>job_name</code>	Logical name added as the <code>job</code> label to every metric scraped by this job

scrape_interval	How often to scrape (can override global per-job)
scrape_timeout	How long to wait before giving up on a scrape
metrics_path	HTTP path to fetch metrics from (default /metrics)
scheme	http (default) or https
static_configs.targets	Explicit host:port list
basic_auth	Send username / password if the target requires it

Applying a config change — step by step

1. Edit the file: `sudo vi /etc/prometheus/prometheus.yml`
2. Pick ONE reload method:
 - Restart manually: Ctrl+C then re-run `./prometheus --config.file=...`
 - Send SIGHUP: `kill -HUP $(pidof prometheus)`
 - systemd: `sudo systemctl restart prometheus`
 - HTTP reload endpoint: `curl -X POST http://localhost:9090/-/reload`
3. Verify in the UI: **Status → Targets** — look for a green "UP".

Best practice

Always run `promtool check config prometheus.yml` (section 2.12) BEFORE reloading, to catch typos without causing downtime.

2.6 Authentication & TLS Encryption

By default, scrapes are plaintext and unauthenticated — anyone who can reach the port can read your metrics. This section secures a Node Exporter target end-to-end.

1. **Generate a TLS cert/key** on the target host (self-signed for testing; CA-signed/Let's Encrypt for production). The cert's CN/SAN must match the hostname Prometheus will use to reach it:

```
sudo openssl req -new -newkey rsa:2048 -days 365 -nodes -x509 \
  -keyout node_exporter.key -out node_exporter.crt \
  -subj "/CN=node" -addext "subjectAltName = DNS:node"
```

2. **Create a web config** for Node Exporter (`/etc/node_exporter/config.yml`):

```
tls_server_config:
  cert_file: node_exporter.crt
  key_file: node_exporter.key
```

3. **Start Node Exporter with that config** (via systemd `ExecStart=... --web.config.file=/etc/node_exporter/config.yml`), then test:

```
curl -k https://localhost:9100/metrics # -k needed only for self-signed certs
```

4. **Copy the cert to the Prometheus server** and point a scrape job at it with TLS verification ON:

```
- job_name: "node"
  scheme: https
  static_configs:
    - targets: ['node:9100']
  tls_config:
    ca_file: /etc/prometheus/node_exporter.crt
    insecure_skip_verify: false # NEVER true in production
```

5. **(Optional) Add Basic Auth.** On the target, generate a bcrypt hash and add it under `basic_auth_users` in the web

config:

```
htpasswd -nBC 12 "" | tr -d ':\n'
# basic_auth_users:
#   prometheus: "$2y$12$...bcrypt hash..."
```

Then on the Prometheus side, add the plaintext password (Prometheus needs the plaintext to authenticate on each scrape):

```
basic_auth:
  username: prometheus
  password: "your_plaintext_password_here"
```

6. **Restart both services** and confirm **Status → Targets** shows UP (not the 401 Unauthorized you'd see mid-migration).

insecure_skip_verify

Setting this `true` disables certificate validation and opens the door to man-in-the-middle attacks. Testing only.

2.7 The Prometheus Data Model (Metrics)

Every metric is: `metric_name{label1="value1", label2="value2"} value`.

```
node_cpu_seconds_total{cpu="0", mode="idle"} 258277.86
```

Time series = one unique combination of metric name + label set. Each new scrape appends a new (timestamp, value) point to that series.

```
node_cpu_seconds_total{cpu="0", instance="server1"}
node_cpu_seconds_total{cpu="1", instance="server1"}
node_cpu_seconds_total{cpu="0", instance="server2"}
node_cpu_seconds_total{cpu="1", instance="server2"}
→ 4 distinct time series from 1 metric name
```

HELP / TYPE metadata

```
# HELP node_disk_discard_time_seconds_total Total seconds spent on all discards.
# TYPE node_disk_discard_time_seconds_total counter
node_disk_discard_time_seconds_total{device="sda"} 0
```

Automatic labels

Prometheus auto-adds `instance` (the scraped `host:port`) and `job` (the job name from config) to every scraped series. Internally the metric name itself is stored as label `__name__`.

```
node_cpu_seconds_total{cpu="0"} == {__name__="node_cpu_seconds_total", cpu="0"}
```

Why use labels instead of separate metric names?

```
# BAD: one metric name per endpoint (hard to sum globally)
requests_auth_total
requests_user_total
requests_cart_total

# GOOD: one metric, a label dimension
requests_total{path="/auth"} 123
requests_total{path="/user"} 456
requests_total{path="/cart"} 78
```

```
sum(requests_total)           # easy total
sum(requests_total) by (path) # easy per-path breakdown
```

High cardinality warning

Every unique label VALUE combination creates a brand-new time series. Never use labels for unbounded values like user IDs, session IDs, or raw request paths — this can exhaust Prometheus's memory/disk.

2.8 The Expression Browser

Prometheus's built-in web UI (default port 9090) for running ad-hoc PromQL, useful for debugging/exploration before wiring a query into a dashboard or alert.

Page	Purpose
Expression input / Graph / Console tabs	Run PromQL, plot results, or see raw table
Status → Configuration	See the exact config Prometheus loaded at startup
Status → Targets	See UP/DOWN state and scrape errors per target
Status → Rules	See loaded recording/alerting rules and their state
Alerts	See active/pending alerts

You can also "time travel": pick a past timestamp in the calendar picker and re-run a query to see historical values, and switch to the **Graph** tab to plot a metric over a time window.

Never expose publicly

The expression browser leaks metrics and config. Restrict with network controls or an auth proxy.

2.9 Running Prometheus in Docker

1. Write `prometheus.yml` on the host as usual.
2. Run the container, bind-mounting the config and mapping port 9090:

```
docker run -d --name prometheus \
  -v /path/to/prometheus.yml:/etc/prometheus/prometheus.yml:ro \
  -v /path/to/data:/prometheus \
  -p 9090:9090 \
  prom/prometheus
```

3. Open `http://localhost:9090`.

Flag	Purpose
<code>-d</code>	Detached (background) mode
<code>-v ...prometheus.yml:...:ro</code>	Bind-mount config read-only
<code>-v ...:/prometheus</code>	Persist TSDB data outside the container
<code>-p 9090:9090</code>	Expose the UI/API on the host

localhost inside a container

`localhost:9090` inside the container refers to the container itself. To scrape services on the *Docker host*, use `host.docker.internal` (Docker Desktop) or `--network=host` (Linux only).

Editing the host `prometheus.yml` updates the file inside the container instantly (bind mount), but Prometheus still needs a reload/restart to pick it up.

2.10 Monitoring Containers (Docker Engine + cAdvisor)

Two complementary metric sources give full-stack container observability:

Source	Gives you	How to enable
Docker Engine metrics	Daemon-level health (CPU, API durations, image counts)	Experimental flag in <code>/etc/docker/daemon.json</code>
cAdvisor	Per-container CPU/memory/filesystem/process metrics	Run as its own privileged container

1. Enable Docker Engine metrics:

```
{ "metrics-addr": "127.0.0.1:9323", "experimental": true }
```

```
sudo systemctl restart docker
curl http://localhost:9323/metrics
```

2. Run cAdvisor:

```
services:
  cAdvisor:
    image: gcr.io/cadvisor/cadvisor
    privileged: true
    volumes:
      - /:/rootfs:ro
      - /var/run:/var/run:ro
      - /sys:/sys:ro
      - /var/lib/docker:/var/lib/docker:ro
    ports:
      - 8080:8080
```

3. Add both as Prometheus scrape jobs:

```
scrape_configs:
  - job_name: "docker"
    static_configs: [{ targets: ["12.1.13.4:9323"] }]
  - job_name: "cadvisor"
    static_configs: [{ targets: ["12.1.13.4:8080"] }]
```

2.11 Real-World Prometheus Use Cases

Use case 1: aggregating metrics across data centers + AWS

Approach	When to use	Trade-off
Per-site Prometheus + federation	Multiple independent sites, occasional cross-site queries	Simple isolation; federation gets complex for global queries
Centralized <code>remote_write</code>	Long-term storage/unified queries (Thanos, Cortex, VictoriaMetrics)	HA + long retention, but adds external storage complexity
Hybrid	Local alerting + long-term analytics	Balances both

Use case 2: alert when memory hits 80%

```
groups:
- name: node.rules
  rules:
  - alert: HighMemoryUsage
    expr: (node_memory_MemTotal_bytes - node_memory_MemAvailable_bytes)
      / node_memory_MemTotal_bytes * 100 > 80
    for: 5m
```

```
labels: { severity: critical }
annotations:
  summary: "High memory usage on {{ $labels.instance }}"
```

Use case 3: find the upload size where latency degrades

1. Instrument the app: a **histogram** for request duration, a gauge/histogram for upload size.
2. Scrape with Prometheus, visualize both series together.
3. Correlate with PromQL:

```
avg_over_time(upload_size_bytes[5m])
  histogram_quantile(0.95, sum(rate(upload_request_duration_seconds_bucket[5m])) by (le))
```

4. Alert only when BOTH are true simultaneously:

```
(avg_over_time(upload_size_bytes[5m]) > 50*1024*1024)
and
(histogram_quantile(0.95, sum(rate(upload_request_duration_seconds_bucket[5m])) by (le)) > 2)
```

2.12 promtool

promtool ships with Prometheus and helps you catch mistakes **before** they cause an outage.

- Validate config files & rule files
- Validate metrics formatting
- Run ad hoc queries against a server
- Debug/profile a running server
- Unit-test recording and alerting rules

```
$ promtool check config /etc/prometheus/prometheus.yml
Checking prometheus.yml
SUCCESS: prometheus.yml is valid prometheus config file syntax
```

If you introduce a typo (e.g. `metric_path` instead of `metrics_path`):

```
$ promtool check config /etc/prometheus/prometheus.yml
FAILED: parsing YAML file prometheus.yml: yaml: unmarshal errors:
  line 24: field metric_path not found in type config.ScrapeConfig
```

Purpose

Run `promtool check config` before every reload/restart. It's the cheapest insurance against a config-typo outage.

Part 3 — PromQL (Prometheus Query Language)

3.1 Introduction to PromQL & Data Types

Purpose

PromQL is how you turn raw stored samples into answers: dashboards, ad-hoc debugging, and alert conditions all boil down to a PromQL expression.

Every PromQL expression returns one of four types:

Type	Meaning
String	A literal string (rarely used directly)
Scalar	A single floating-point number, e.g. 54.743
Instant vector	One sample per matching time series, all at the same timestamp
Range vector	Multiple samples per matching time series, across a time window

Instant vector example

```
$ node_cpu_seconds_total
node_cpu_seconds_total{cpu="0", instance="server1"} 258277.86
node_cpu_seconds_total{cpu="1", instance="server1"} 448430.21
node_cpu_seconds_total{cpu="0", instance="server2"} 941202.32
node_cpu_seconds_total{cpu="1", instance="server2"} 772838.83
→ 4 time series, ONE value each, all at the same instant
```

Range vector example

```
$ node_cpu_seconds_total[3m]
→ each matching series now returns MULTIPLE (timestamp, value) points
   covering the last 3 minutes (however many scrapes fell in that window)
```

Instant vector: one dot per series, same time



Range vector: many dots per series, over a window [3m]



3.2 Selectors and Label Matchers

A bare metric name with no filter returns every time series that has that name. Use **label matchers** inside {...} to narrow it down.

Operator	Meaning	Example
=	Equality	node_filesystem_avail_bytes{instance="node1"}
!=	Inequality (exclude)	node_filesystem_avail_bytes{device!="tmpfs"}
=~	Regex match	node_filesystem_avail_bytes{device=~"/dev/sda.*"}
!~	Negative regex	node_filesystem_avail_bytes{mountpoint!~"/boot.*"}

Combine matchers with a comma (implicit AND):

```
node_filesystem_avail_bytes{instance="node1", device!="tmpfs"}
```

Add a range selector in square brackets to turn any instant selector into a range vector:

```
node_arp_entries{instance="node1"}[2m]
→ [8@t1, 2@t2, 3@t3, 1@t4, 7@t5, 7@t6, 7@t7, 6@t8] (all samples in the last 2 minutes)
```

3.3 Modifiers: offset and @

Purpose

By default a query returns the LATEST value. These modifiers let you look into the past — essential for incident postmortems ("what was the value at 06:06 yesterday?").

offset — shift relative to now

```
node_memory_Active_bytes{instance="node1"} offset 5m # value from 5 minutes ago
node_memory_Active_bytes{instance="node1"} offset 5d # 5 days ago
node_memory_Active_bytes{instance="node1"} offset 1h30m # 1.5 hours ago
```

Suffix	Meaning
ms	milliseconds
s	seconds
m	minutes
h	hours
d	days
w	weeks
y	years

@ — anchor to an exact Unix timestamp

```
node_memory_Active_bytes{instance="node1"} @1663265188
```

Combining both

```
node_memory_Active_bytes{instance="node1"} @1663265188 offset 5m
node_memory_Active_bytes{instance="node1"} offset 5m @1663265188
→ both return the value from 5 minutes BEFORE that timestamp (order doesn't matter)
```

When you'd actually use this

Dashboards like Grafana translate their time-range picker into these modifiers automatically. You only type `offset / @` yourself when querying the raw Expression Browser or HTTP API directly.

3.4 PromQL Practice Demo (Putting Selectors + Modifiers Together)

```
# All series for CPU 0
node_cpu_seconds_total{cpu="0"}

# CPU 0 AND idle mode
```

```
node_cpu_seconds_total{cpu="0", mode="idle"}

# regex: mountpoints starting with /run
node_filesystem_files{mountpoint=~"^/run.*"}

# regex + range vector: last 5 min of history for mountpoints NOT starting with /run
node_filesystem_files{mountpoint!~"^/run.*"}[5m]

# 5 minutes of data, starting 30 minutes ago
node_filesystem_files{mountpoint!~"^/run.*"}[5m] offset 30m

# value at an exact historical timestamp
node_filesystem_files{mountpoint="/" } @ 1620000000
```

3.5 Operators

Arithmetic: + - * / % ^

```
node_memory_active_bytes{instance="node1"}           # 2204815360
node_memory_active_bytes{instance="node1"} / 1024    # 2153168 (bytes -> KiB)
```

Arithmetic on a metric produces a new vector that keeps the labels but DROPS the original metric name (`__name__`).

Comparison: == != > < >= <=

```
node_network_flags > 100
→ keeps ONLY series whose value is > 100 (this FILTERS the vector, it doesn't return true/false)
```

The bool modifier

Turns a comparison from a *filter* into a *0/1 indicator* for every input series — essential for alert expressions that need a clean numeric signal.

```
node_filesystem_avail_bytes < bool 1000
→ every series is KEPT; value becomes 1 (true) or 0 (false)
```

Operator precedence

Same rules as normal math: `2 * 3 % 2` = `(2*3)%2` (left-to-right); `2 ^ 3 ^ 2` = `2^(3^2)` (right-associative, power only). Use parentheses whenever in doubt.

Logical / set operators: and, or, unless

Operator	Behavior
and	Intersection: series present in BOTH sides
or	Union: series present in EITHER side
unless	Set difference: left-side series NOT matched on the right

```
# Intersection: available bytes between 1000 and 3000
node_filesystem_avail_bytes > 1000 and node_filesystem_avail_bytes < 3000

# Union: below 500 OR above 70000
node_filesystem_avail_bytes < 500 or node_filesystem_avail_bytes > 70000

# Difference: >1000 but NOT >30000
node_filesystem_avail_bytes > 1000 unless node_filesystem_avail_bytes > 30000
```

3.6 Vector Matching (on, ignoring, group_left, group_right)

Purpose

When you divide/combine TWO different metrics, Prometheus needs a rule for which sample on the left pairs with which sample on the right. This section is the #1 source of "why did my query return nothing?" confusion.

Default rule: exact label match

```
node_filesystem_avail_bytes / node_filesystem_size_bytes * 100
→ works ONLY if both sides have IDENTICAL label sets (device, instance, mountpoint, ...)
```

on(...) / ignoring(...) — control which labels must match

```
# errors has an extra "code" label that requests doesn't have:
http_errors{code="500"} / ignoring(code) http_requests
# same result, expressed the other way:
http_errors{code="500"} / on(method) http_requests
```

Only series present on BOTH sides survive; anything unmatched is silently dropped.

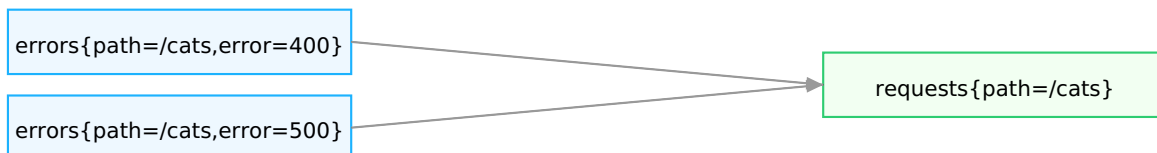
group_left / group_right — many-to-one and one-to-many

By default PromQL requires strict one-to-one matching and will ERROR with "multiple matches for labels" if one right-hand sample would match several left-hand samples.

```
# http_errors has MULTIPLE rows per path (one per error code);
# http_requests has ONE row per path -> classic many-to-one
http_errors / on(path) group_left() http_requests
→ group_left() tells PromQL "it's OK, the LEFT side has many matches per right-hand row"
→ it also PRESERVES the left side's extra label (error) in the result
```

Use `group_right()` when it's the RIGHT side that has multiple matches instead.

Many-to-one with group_left()



TWO left rows match ONE right row → needs group_left()

Matching mode	Description	Example
One-to-one (default)	Each left sample matches at most one right sample	A / B
Many-to-one	Multiple left rows → one right row	A / on(path) group_left() B
One-to-many	One left row → multiple right rows	A / on(path) group_right() B

3.7 Aggregation Operators (by, without)

Purpose

Aggregation collapses MANY time series into FEWER, e.g. "total requests across all instances" instead of one line per instance.

Operator	Description
<code>sum()</code>	Sum of all matching series
<code>max()</code> / <code>min()</code>	Highest / lowest value
<code>avg()</code>	Average
<code>count()</code>	Number of series

```
$ sum(http_requests)           # {} 41 -- ALL labels collapsed away
$ sum by(path) (http_requests) # keep 'path', collapse everything else
{path="/auth"} 4
{path="/user"} 12
$ sum without(cpu, mode) (node_cpu_seconds_total)
→ drops cpu & mode, keeps every other label (e.g. instance)
```

`by(...)` vs `without(...)`

`by(x)` = "keep ONLY label x, drop everything else." `without(x)` = "drop label x, keep everything else." They are inverses — pick whichever is shorter to write for your case.

Aggregation PERMANENTLY drops any label not listed in `by(...)` (or dropped by `without(...)`). If you need that label later, don't aggregate it away too early.

3.8 PromQL Functions

Category	Purpose	Examples
Math	Per-sample numeric transforms	<code>ceil(v)</code> , <code>floor(v)</code> , <code>abs(v)</code>
Time/date	Extract parts of the evaluation time	<code>time()</code> , <code>hour()</code> , <code>day_of_week()</code>
Conversions	scalar ↔ instant vector	<code>scalar(v)</code> , <code>vector(s)</code>
Sorting	Order results by value	<code>sort(v)</code> , <code>sort_desc(v)</code>
Presence checks	Detect missing/present series	<code>absent(v)</code> , <code>present_over_time(v[r])</code>
Counter rates	Convert increasing counters → per-second rate	<code>rate(v[r])</code> , <code>irate(v[r])</code>

`rate()` vs `irate()` — the most important function pair

```
# rate(): AVERAGE per-second rate over the whole window
#   samples in the 1m window: 1.2, 2.3, 3.1, 3.3
#   rate = (last - first) / window_seconds = (3.3-1.2)/60 = 0.035/s

# irate(): INSTANT slope using ONLY the last two samples
#   irate = (last - second_last) / (time between them)
```

Function	Behavior	Best for
<code>rate(v[r])</code>	Averages across the entire range (handles counter resets)	Stable — alerting, dashboards
<code>irate(v[r])</code>	Instant slope from the last 2 samples	Responsive graphs of volatile counters

Correct aggregation order

Always `rate()` FIRST, per series, THEN `sum()` — never the reverse, or counter-reset handling breaks.

```
sum without(code, handler) (rate(http_requests_total[24h])) # correct order
```

Presence checks

```
absent(nonexistent{job="myjob"})           # returns {} if series exists, else 1 element with value 1
absent_over_time(node_memory_Active_bytes[1h]) # 1 if NO samples in range
present_over_time(node_filesystem_avail_bytes[5m]) # 1 if AT LEAST ONE sample in range
```

These power "is my exporter dead" alerts: `absent(up{job="node"})` fires when a whole job vanishes.

Real example: network throughput

```
rate(node_network_transmit_bytes_total[1m]) # turns a raw ever-increasing byte counter
                                             # into bytes-per-second throughput
```

3.9 Subqueries

Purpose

`max_over_time()`, `avg_over_time()`, etc. need a RANGE vector as input. But `rate(...)` returns an INSTANT vector. A subquery bridges that gap by re-evaluating an instant expression repeatedly across a historical window.

```
# THIS FAILS: rate() is an instant vector, max_over_time() wants a range vector
max_over_time(rate(http_requests_total[10m]))

# Correct: subquery syntax = <instant_query> [<range>:<resolution>]
max_over_time(rate(http_requests_total[1m])[5m:30s])
#           inner [1m] = rate()'s own sample window
#           outer [5m:30s] = re-evaluate rate() every 30s over the last 5m
```

Vector type	Represents	Typical use
Instant vector	1 value per series, 1 timestamp	<code>rate(...)</code> , plain metric selectors
Range vector	Many values per series over a window	<code>max_over_time(...)</code> , subqueries like <code>rate(...)[5m:30s]</code>

Very small subquery resolutions are expensive — they multiply the number of evaluations Prometheus must perform. Pick a resolution matched to the precision you actually need.

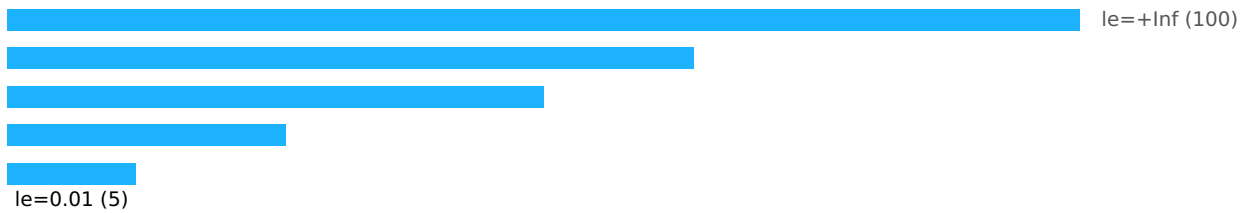
3.10 Histograms & Summaries + `histogram_quantile()`

A histogram metric (e.g. `request_latency_seconds`) actually exposes THREE series:

Submetric	Meaning
<code>_count</code>	Total number of observations (a counter)
<code>_sum</code>	Sum of all observed values (→ lets you compute averages)
<code>_bucket{le="X"}</code>	Cumulative count of observations $\leq X$

```
request_latency_seconds_bucket{le="0.05"} 50 # 50 requests were <= 0.05s
request_latency_seconds_bucket{le="0.1"} 100 # 100 requests were <= 0.1s (INCLUDES the 50 above)
request_latency_seconds_bucket{le="+Inf"} 100 # must equal _count, or instrumentation is buggy
request_latency_seconds_count 100
request_latency_seconds_sum 5.034496069
```

Cumulative buckets (each bar INCLUDES everything to its left)



Common queries

```
# request rate
rate(request_latency_seconds_count[1m])

# AVERAGE latency (sum/count trick, NOT a percentile)
rate(request_latency_seconds_sum[5m]) / rate(request_latency_seconds_count[5m])

# fraction of requests <= 0.06s
rate(request_latency_seconds_bucket{le="0.06"}[5m])
  / ignoring(le) rate(request_latency_seconds_count[5m])

# observations strictly between two boundaries
increase(request_latency_seconds_bucket{le="0.05"}[5m])
  - increase(request_latency_seconds_bucket{le="0.02"}[5m])
```

histogram_quantile() — estimating percentiles

```
histogram_quantile(0.95, request_latency_seconds_bucket{path="/cars"})
# When aggregating multiple instances, sum by (le) FIRST:
histogram_quantile(0.95, sum(rate(request_latency_seconds_bucket[5m])) by (le))
```

It's an ESTIMATE

`histogram_quantile()` linearly interpolates between bucket boundaries — it is not exact. If your SLO cares about a specific threshold (e.g. 0.5s), make sure you have a bucket boundary AT or very near that threshold, or the estimate can be misleading.

Histogram vs Summary

Aspect	Histogram	Summary
Quantiles	Server-side, ANY quantile via <code>histogram_quantile()</code>	Client-side, must pre-choose which quantiles to expose
Aggregation across instances	Easy — sum buckets by <code>le</code>	Hard — per-instance quantiles don't combine
Cardinality cost	Higher (one series per bucket)	Lower on server, more client CPU
Best for	Flexible, cross-instance percentiles	Exact, known quantiles, single-instance

3.11 Recording Rules

Purpose

Recording rules pre-compute an expensive/frequent PromQL expression on a schedule and save the RESULT as a new metric — so dashboards read a cheap pre-computed series instead of recalculating it every time.

```
# prometheus.yml
rule_files:
  - /etc/prometheus/rules/*.yaml
```

```
# rules.yml
groups:
- name: node
  interval: 15s
  rules:
    - record: node_memory_memFree_percent
      expr: 100 - (100 * node_memory_MemFree_bytes / node_memory_MemTotal_bytes)
    - record: node_filesystem_free_percent
      expr: 100 * node_filesystem_free_bytes{job="node"} / node_filesystem_size_bytes{job="node"}
    - record: node_filesystem_free_percent_avg      # can reference a PRIOR rule in the same group
      expr: avg by(instance)(node_filesystem_free_percent)
```

Field	Purpose
<code>groups[].interval</code>	Optional; overrides the global evaluation interval for this group
<code>record</code>	Name of the NEW metric to store
<code>expr</code>	PromQL expression to evaluate
<code>labels</code>	Optional labels to add/override

Groups run in parallel, RULES within a group run in order

If rule B references rule A's recorded metric, A must be listed BEFORE B in the same group.

Naming convention

Structure: `level:metric:operation` (or underscore-joined). Example: an HTTP errors counter aggregated by job/method/path with a 5-minute rate → `job_method_path_http_errors_rate_5m`. Group all rules for one job together for consistent evaluation intervals.

Applying and verifying

1. Create/edit the rule file.
2. Ensure `rule_files` in `prometheus.yml` points to it.
3. Reload: `curl -X POST http://localhost:9090/-/reload` or restart.
4. Verify: **Status → Rules** in the UI shows `OK`; query the new metric name directly to confirm it exists.

3.12 The HTTP API

Purpose

Everything the Web UI does is really just calling this API — use it directly for automation, scripts, and integrations like Grafana.

Endpoint	Purpose	Required params
<code>POST /api/v1/query</code>	Instant query (current value)	<code>query</code>
<code>POST /api/v1/query</code>	Instant query at a past time	<code>query</code> , <code>time</code>
<code>POST /api/v1/query_range</code>	Multi-point series over an interval	<code>query</code> , <code>start</code> , <code>end</code> , <code>step</code>

Instant query

```
curl 'http://localhost:9090/api/v1/query' \
  --data 'query=node_cpu_seconds_total{job="node"}' | jq
```

```
{
  "status": "success",
  "data": {
```

```
"resultType": "vector",
"result": [
  { "metric": { "__name__": "node_cpu_seconds_total", "cpu": "0", "job": "node", "mode": "idle"},
    "value": [1670382765.652, "502077.09"] }
]
}
```

Instant query at a specific timestamp

```
curl 'http://localhost:9090/api/v1/query' \
--data 'query=node_memory_Active_bytes{job="node"}' \
--data 'time=1670380680.132' | jq
```

Range query

```
curl 'http://localhost:9090/api/v1/query_range' \
--data 'query=node_memory_Active_bytes{job="node"}' \
--data 'start=1670380080.132' \
--data 'end=1670380680.132' \
--data 'step=15' | jq
→ returns a "matrix" result: each series has a list of [timestamp, value] pairs
```

When using shell `curl`, wrap the whole `--data` value in single quotes and use double quotes inside the PromQL label selector to avoid quoting conflicts.

Part 4 — Dashboarding & Visualization

4.1 Visualization Options Overview

Purpose

Raw numbers are hard to spot trends in; plotting them reveals patterns (spikes, slow drifts, correlations) that are invisible in a table of numbers.

Tool	Type	Best for
Expression Browser	Built-in	Ad-hoc PromQL exploration/debugging
Console Templates	Built-in	Small, fixed HTML pages with a few embedded charts
Grafana	Third-party	Rich, interactive, shareable team dashboards

Use the Expression Browser to troubleshoot and validate a query, Console Templates for a small curated set of views with no external dependency, and Grafana for production, long-lived, shared dashboards with templating and alerting UI.

4.2 Console Templates

Console templates are plain HTML files augmented with the Go templating language (`{{ ... }}`), stored (by default) in `/etc/prometheus/consoles` , and served at `http://<host>:9090/consoles/<file>` .

File	Purpose
<code>index.html.example</code>	Example index linking the other pages
<code>node-cpu.html</code>	Node CPU charts
<code>node-overview.html</code>	Comprehensive node dashboard
<code>prometheus-overview.html</code>	Prometheus server's own metrics

Building a custom console page, step by step

1. Create the file: `sudo vi /etc/prometheus/consoles/demo.html`
2. Wrap your content with the standard fragments for consistent CSS/JS/layout:

```
{{ template "head" . }}
{{ template "prom_content_head" . }}
<!-- your content here -->
{{ template "prom_content_tail" . }}
{{ template "tail" }}
```

3. Add a clickable metric drilldown and a live chart:

```
<h1>Memory details</h1>
<p>Active memory:
  {{ template "prom_query_drilldown" (args "node_memory_Active_bytes") }}
</p>
<div id="graph"></div>
<script>
  new PromConsole.Graph({
    node: document.querySelector("#graph"),
    expr: "node_memory_Active_bytes"
  });
</script>
```

4. Open `http://<prometheus-host>:9090/consoles/demo.html` to see the header/footer, the clickable metric value, and a rendered time-series chart.

When to reach for this

Console templates are utilitarian but zero-dependency — good for a small number of curated views baked right into the Prometheus server itself. For anything richer (templating variables, alert overlays, team sharing), use Grafana instead.

Part 5 — Application Instrumentation

5.1 Client Libraries: Introduction

Purpose

Exporters cover infrastructure metrics, but only YOUR application code knows business-level facts (which endpoint was hit, how long the DB call took, how many items are queued). A **client library** lets you define those metrics in code and exposes them at `/metrics` for Prometheus to scrape.

Language	Official library
Go	<code>client_golang</code>
Java/Scala	<code>client_java</code>
Python	<code>prometheus_client</code>
Ruby	<code>prometheus-client</code>

```
from prometheus_client import Counter, Gauge, Histogram, start_http_server
from time import sleep
from random import random

REQUESTS = Counter('myapp_requests_total', 'Total HTTP requests')
IN_PROGRESS = Gauge('myapp_inprogress_requests', 'In-progress requests')
REQUEST_LATENCY = Histogram('myapp_request_duration_seconds', 'Request latency seconds')

start_http_server(8000) # exposes /metrics on port 8000

def handle_request():
    REQUESTS.inc()
    IN_PROGRESS.inc()
    with REQUEST_LATENCY.time():
        sleep(random() * 0.5)
    IN_PROGRESS.dec()
```

5.2 Instrumentation Basics (Flask + Counter)

1. Start with a minimal Flask app:

```
from flask import Flask
app = Flask(__name__)

@app.get("/cars")
def get_cars():
    return ["toyota", "honda", "mazda", "lexus"]

if __name__ == "__main__":
    app.run(port=5001)
```

2. Install the client: `pip install prometheus_client`
3. Create a `Counter` and increment it inside the handler:

```
from prometheus_client import Counter
REQUESTS = Counter("http_requests_total", "Total number of requests")

@app.get("/cars")
def get_cars():
    REQUESTS.inc()
    return [...]
```

4. Expose the metrics endpoint — two options:

Option	How	Trade-off
Separate server	<code>start_http_server(8000)</code>	Simple; metrics port is independent of app port
Same port (WSGI middleware)	<code>DispatcherMiddleware(app.wsgi_app, {"/metrics": make_wsgi_app()})</code>	One port to manage/secure

```
from prometheus_client import make_wsgi_app
from werkzeug.middleware.dispatcher import DispatcherMiddleware
app.wsgi_app = DispatcherMiddleware(app.wsgi_app, {"/metrics": make_wsgi_app()})
```

5. Count EVERY request without repeating `.inc()` in every handler, using a global hook:

```
@app.before_request
def increment_request_counter():
    if request.path != "/metrics": # don't count scrapes of /metrics itself!
        REQUESTS.inc()
```

`start_http_server` creates a SEPARATE HTTP server (different port) — Flask's `before_request` hook will NOT fire for it. Use the WSGI middleware approach if you want everything on one port and one set of hooks.

5.3 Labels in Instrumentation

Purpose

Without labels you'd need a separate metric NAME per endpoint (`requests_cars_total` , `requests_boats_total` ...) — painful to maintain and impossible to sum globally without hardcoding every name. Labels solve this with ONE metric name and a dimension.

```
from prometheus_client import Counter
REQUESTS = Counter("http_requests_total", "Total number of requests", ["path"])

@app.get("/cars")
def get_cars():
    REQUESTS.labels("/cars").inc()
    return [...]

@app.get("/boats")
def get_boats():
    REQUESTS.labels("/boats").inc()
    return [...]
```

```
$ http_requests_total
http_requests_total{path="/cars"} 5.0
http_requests_total{path="/boats"} 2.0
$ sum(http_requests_total) # 7.0 -- trivial global total
```

Multiple labels — order matters, it must match the order declared:

```
REQUESTS = Counter("http_requests_total", "...", ["path", "method"])
REQUESTS.labels("/cars", "GET").inc()
```

Approach	Pros	Cons
One metric name per path	Simple to implement	Hard to aggregate; error-prone as endpoints grow
One metric + labels (recommended)	Easy aggregation, flexible querying	Risk of high cardinality if a label is unbounded

Cardinality

Never label with user IDs, session tokens, or raw request IDs — keep labels to a small, well-defined set of values.

5.4 Histogram & Summary Instrumentation

```
from prometheus_client import Histogram
import time
from flask import Flask, request
app = Flask(__name__)

LATENCY = Histogram('request_latency_seconds', 'Request Latency', labelnames=['path', 'method'])

def before_request():
    request.start_time = time.time()

def after_request(response):
    request_latency = time.time() - request.start_time
    LATENCY.labels(request.path, request.method).observe(request_latency)
    return response

if __name__ == '__main__':
    start_http_server(8000)
    app.before_request(before_request)
    app.after_request(after_request)
    app.run()
```

Customizing buckets

```
LATENCY = Histogram('request_latency_seconds', 'Flask Request Latency',
                    labelnames=['path', 'method'],
                    buckets=[0.01, 0.02, 0.03, 0.04, 0.05, 0.06, 0.07, 0.08, 0.09, 0.1])
```

Use finer buckets for low-latency apps, wider buckets for high-latency ones — match the distribution you actually expect.

Using a Summary instead

```
from prometheus_client import Summary
LATENCY_SUMMARY = Summary('request_latency_seconds', 'Flask Request Latency', labelnames=['path', 'method'])
LATENCY_SUMMARY.labels(request.path, request.method).observe(request_latency)
```

The Python client does NOT support configurable server-computed quantiles for Summary the way some other languages do. Prefer Histogram + `histogram_quantile()` in Prometheus unless you specifically need client-side quantiles.

5.5 The Gauge Metric Type

Purpose

Use a Gauge for anything that goes UP and DOWN — in-progress requests, queue length, free memory.

```
from prometheus_client import Gauge
IN_PROGRESS = Gauge('inprogress_requests', 'Total number of requests in progress',
                    labelnames=['method', 'path'])

def before_request():
    IN_PROGRESS.labels(request.method, request.path).inc()

def after_request(response):
    IN_PROGRESS.labels(request.method, request.path).dec()
    return response
```

Method	Effect
<code>inc()</code>	Increment by 1 (or a given amount)

<code>dec()</code>	Decrement by 1 (or a given amount)
<code>set(value)</code>	Set the gauge to an exact value

Never let inc/dec go unpaired

If an exception skips your `dec()`, the gauge inflates forever and misleads dashboards/alerts. Use `try/finally` or a framework teardown hook:

```
def handle_request():
    IN_PROGRESS.labels(request.method, request.path).inc()
    try:
        ... # process
    finally:
        IN_PROGRESS.labels(request.method, request.path).dec()
```

Metric Type	Use Case	Behavior
Gauge	Current value that fluctuates	<code>inc()</code> , <code>dec()</code> , <code>set()</code>
Counter	Cumulative, only increases	<code>inc()</code> only

5.6 Naming & Instrumentation Best Practices

Naming rules

- `snake_case`: lowercase, underscores.
- Prefix with app/library name, e.g. `postgres_connection_errors`.
- Always include a base unit — `seconds`, `bytes` — never `milliseconds` / `kilobytes`.
- Counters MUST end in `_total`. Never manually add `_sum` / `_bucket` / `_count` — the client library generates those automatically for histograms/summaries.

```
# GOOD
process_cpu_seconds
http_requests_total
redis_connection_errors
node_disk_read_bytes_total

# BAD and why
container_docker_restarts # wrong prefix order -> docker_container_restarts
http_requests_sum         # never hand-add _sum
nginx_disk_free_kilobytes # use base unit: bytes
dotnet_queue_waiting_time # missing unit suffix -> _seconds
```

Suffix	Metric type	Notes
<code>_total</code>	Counter	You add this yourself
<code>_count</code> , <code>_sum</code> , <code>_bucket</code>	Histogram/Summary	Auto-generated — don't add manually

What to instrument, by system type

System type	What to track
Online serving (APIs, DBs)	Total requests, errors, latency (histogram), in-progress (gauge)
Offline processing (workers)	Items processed (counter), queue/backlog size (gauge), work-in-progress (gauge), errors per stage
Batch jobs (cron-style)	Per-stage time, overall runtime, success/failure counts, items processed — pushed via Pushgateway since they may finish before a scrape

Do not use Pushgateway for ephemeral per-request metrics — only for the whole batch job's summary metrics.

Part 6 — Service Discovery

6.1 Why Service Discovery?

Purpose

A hand-maintained `static_configs` list works for a handful of stable servers, but in Kubernetes/cloud auto-scaling environments, instances appear and disappear constantly. Service discovery lets Prometheus automatically track WHO to scrape, without manual edits.

```
# A static list -- the "manual" baseline
scrape_configs:
  - job_name: "web"
    static_configs: [{ targets: ["localhost:9090"] }]
  - job_name: "node"
    static_configs: [{ targets: ["192.168.1.168:9100"] }]
```

Discovery type	Typical use case
kubernetes_sd	Pods/services/endpoints in a cluster, updated as pods scale/move
ec2_sd	AWS EC2 instances, filtered by tags/metadata
consul_sd	Services registered in HashiCorp Consul
gce_sd / azure_sd	GCP / Azure VM discovery via metadata
dns_sd	Simple DNS A/SRV record based discovery
file_sd	External process writes a target list file that Prometheus watches

6.2 File-based Service Discovery

The simplest, most portable discovery mechanism — useful as a lightweight static-style approach, or to bridge a discovery system Prometheus doesn't support natively.

1. Write a JSON (or YAML) targets file:

```
[
  { "targets": ["node1:9100", "node2:9100"], "labels": {"team":"dev","job":"node"} },
  { "targets": ["localhost:9090"], "labels": {"team":"monitoring","job":"prometheus"} }
]
```

2. Reference it from `prometheus.yml`, with optional globs:

```
scrape_configs:
  - job_name: file-example
    file_sd_configs:
      - files: ['file-sd.json', '*.json']
```

3. Prometheus **watches these files automatically** and reloads targets when they change — no restart needed.
4. Verify under **Status → Service Discovery** in the Web UI.

Treat these files as sensitive — don't store credentials in them, and set proper file permissions.

6.3 AWS EC2 Service Discovery

1. **Create an IAM user** for Prometheus with programmatic access and the **AmazonEC2ReadOnlyAccess** managed

policy attached (or better: use an EC2 instance role in production).

2. Configure the scrape job:

```
scrape_configs:
- job_name: "ec2"
  ec2_sd_configs:
  - region: us-east-1
    access_key: AKIAQS0UAYMSDJI177
    secret_key: G2y3/xjKr0m0sWmA0SWh/VCWCBR/ShmqMJHRJkoX1h
```

If Prometheus itself runs on an EC2 instance with an appropriate role, omit `access_key` / `secret_key` entirely.

3. Inspect the discovered metadata (available for relabeling — see 6.4):

```
meta_ec2_instance_id="i-0e23a196c0a198db"
meta_ec2_private_ip="172.31.1.156"
meta_ec2_public_ip="3.80.117.102"
meta_ec2_instance_type="t2.micro"
meta_ec2_availability_zone="us-east-1b"
meta_ec2_tag_instance="web"
```

4. Validate: Status → Service Discovery shows discovered instances; Status → Targets shows reachability. "Down" usually means a security-group/VPC-routing issue.

Never commit AWS keys to version control. Prefer an instance role, environment credentials, or a secrets manager.

VPC note

By default Prometheus scrapes the instance's PRIVATE IP, assuming it runs inside the same network. If Prometheus is outside the VPC, relabel `meta_ec2_public_ip` into `__address__`, or run Prometheus inside the VPC instead.

6.4 Relabeling

Purpose

Discovery gives you raw metadata (e.g. `__meta_ec2_tag_env`). Relabeling lets you filter which discovered targets to actually scrape, and turn that metadata into normal, permanent target/metric labels.

Phase	Config block	Sees	Typical use
Pre-scrape	<code>relabel_configs</code>	Discovery <code>__meta_*</code> labels + target labels	Filter targets, map metadata → target labels
Post-scrape	<code>metric_relabel_configs</code>	Scraped metric names/labels (incl. <code>__name__</code>)	Drop/rename metrics, remove labels

Important

`__meta_*` labels are DISCARDED after relabeling unless you explicitly map them to a normal target label. Labels starting/ending with `__` are always internal-only.

Filter targets by a tag — keep / drop

```
relabel_configs:
- source_labels: [__meta_ec2_tag_env]
  regex: prod
  action: keep      # only scrape targets tagged env=prod

- source_labels: [__meta_ec2_tag_env]
  regex: dev
  action: drop      # exclude targets tagged env=dev
```

Extract part of a label with replace

```
# turn __address__ "172.31.1.156:80" into a new label "ip" = "172.31.1.156"
relabel_configs:
  - source_labels: [__address__]
    regex: '(.*)\:.*'
    target_label: ip
    action: replace
    replacement: '$1'
```

labeldrop / labelkeep / labelmap

```
# drop one specific label by name
- regex: __meta_ec2_owner_id
  action: labeldrop

# keep ONLY instance and job, drop everything else
- regex: instance|job
  action: labelkeep

# rename ALL __meta_ec2_* keys into ec2_* target labels (keeps values)
- regex: '__meta_ec2_(.*)'
  action: labelmap
  replacement: 'ec2_$1'
```

metric_relabel_configs — post-scrape

```
# drop an entire metric
metric_relabel_configs:
  - source_labels: [__name__]
    regex: http_errors_total
    action: drop

# rename a metric
- source_labels: [__name__]
  regex: http_errors_total
  action: replace
  target_label: __name__
  replacement: http_failures_total

# drop a metric-level label from everything
- regex: code
  action: labeldrop
```

Action	Purpose
keep	Retain only matching targets/metrics (anything else is dropped)
drop	Exclude matching targets/metrics
replace	Set/overwrite a label (or the metric name)
labeldrop	Remove label(s) matching a regex
labelkeep	Keep only label(s) matching a regex
labelmap	Rename label KEYS via regex/replacement

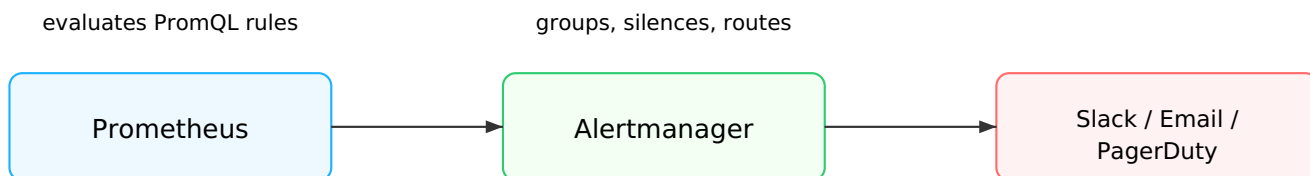
Relabel rules run sequentially. An unqualified `action: keep` DROPS every target that doesn't match — a common source of "why did my targets disappear?" bugs. Always verify in **Status → Targets** after changes.

Part 7 — Alerting

7.1 Alerting Overview & Alert Lifecycle

Purpose

Dashboards only help if someone is looking at them. Alerting ensures the right person is notified automatically when a PromQL condition becomes true — e.g. disk full, node down, latency too high.



Key separation of concerns

Prometheus **evaluates** conditions and generates alert instances. It does NOT deliver notifications. **Alertmanager** receives those alerts and handles dedup, grouping, silencing, routing, and actual delivery.

How an alert is generated

```
node_filesystem_avail_bytes < 1000
```

If this query matches one time series, Prometheus creates ONE alert instance. If it matches N series, it creates N separate alert instances — one per matching series.

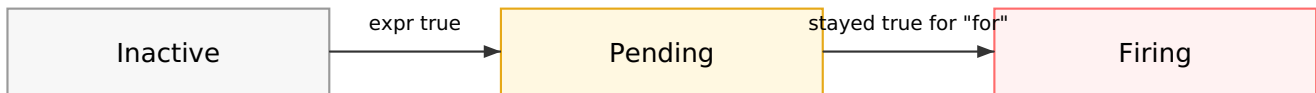
Alert rules live alongside recording rules

```
groups:
- name: node
  interval: 15s
  rules:
    - record: node_memory_memFree_percent
      expr: 100 * node_memory_MemFree_bytes{job="node"} / node_memory_MemTotal_bytes{job="node"}
    - alert: LowMemory
      expr: node_memory_memFree_percent < 20
      for: 3m
    - alert: NodeDown
      expr: up{job="node"} == 0
      for: 5m
```

`for` delays the transition to "firing" until the condition has been continuously true for that duration — this avoids false positives from brief blips.

For extremely urgent conditions (data corruption, total outage) keep `for` short or omit it so notification isn't delayed.

Alert lifecycle



State	Meaning
Inactive	Expression returns no results
Pending	Expression is true but <code>for</code> hasn't elapsed yet
Firing	True for the full <code>for</code> duration — sent to Alertmanager

Field	Purpose
<code>alert</code>	Alert name
<code>expr</code>	The PromQL condition
<code>for</code>	Delay before becoming firing
<code>labels</code>	Identity used for routing/grouping (Alertmanager)
<code>annotations</code>	Human-readable notification text

7.2 Labels vs Annotations

Purpose

Labels decide WHO gets the alert (identity/routing); annotations decide WHAT the message actually says (human-readable content). Mixing these up is a common mistake — annotations can never be used for routing.

```

groups:
- name: node
  rules:
    - alert: NodeDown
      expr: up{job="node"} == 0
      labels:
        severity: warning
    - alert: MultipleNodesDown
      expr: avg_without(instance)(up{job="node"}) <= 0.5
      labels:
        severity: critical
  
```

```

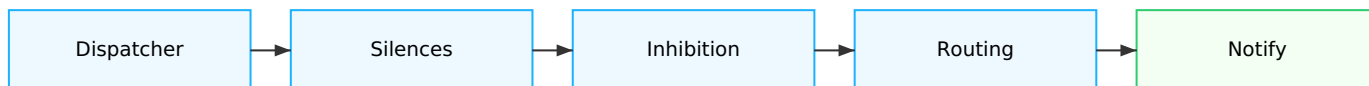
- alert: node_filesystem_free_percent
  expr: 100 * node_filesystem_free_bytes{job="node"} / node_filesystem_size_bytes{job="node"} < 70
  annotations:
    description: "filesystem {{ .Labels.device }} on {{ .Labels.instance }} is low, available: {{ .Value }}"
  
```

Field	Common template
<code>{{ .Labels }}</code>	Full label map
<code>{{ .Labels.instance }}</code>	A specific label value
<code>{{ .Value }}</code>	The metric value that triggered the alert

Concept	Purpose	Example
Labels	Alert identity, routing, grouping, dedup, inhibition	<code>severity: critical</code>
Annotations	Human-readable notification content, templated	<code>description: "disk on {{ .Labels.instance }} is low"</code>

7.3 Alertmanager Architecture

Alerts flow through five sequential stages:



Stage	What it does
Dispatcher / grouping	Batches related alerts (by <code>alertname</code> , <code>instance</code> , etc.) into one notification
Silences	Applies temporary mutes for known maintenance windows
Inhibition	Suppresses alert Y if a related alert X is firing (avoids noisy cascades)
Routing	Matches alert labels against the routing tree to pick the receiver(s)
Notification engine	Formats and delivers to email/Slack/PagerDuty/webhooks, retries on failure

Silences vs Inhibition

Silences = an explicit, time-boxed mute you create manually. Inhibition = automatic, conditional suppression when a related "source" alert is already firing (and not itself silenced).

7.4 Installing Alertmanager

1. Download & extract:

```
wget <url-to-alertmanager-tarball>
tar xvf alertmanager-*.tar.gz
cd alertmanager-*/
```

2. Start it: `./alertmanager` — default port **9093**, config file `alertmanager.yml` in the current directory.
3. Point Prometheus at it, in `prometheus.yml` :

```
alerting:
  alertmanagers:
    - static_configs:
      - targets: ["alertmanager1:9093", "alertmanager2:9093"]
```

4. Restart Prometheus and open `http://localhost:9093` to confirm.

File	Purpose
<code>alertmanager</code>	The executable
<code>alertmanager.yml</code>	Routing/receivers config
<code>amtool</code>	CLI utility (manage silences, inspect config)
<code>data/</code>	Silences and notification state storage

7.5 Alertmanager Configuration (routing tree)

Three sections: `global` (shared defaults), `route` (the routing tree), `receivers` (notification destinations).

```
global:
  smtp_smarthost: 'mail.example.com:25'
  smtp_from: 'test@example.com'

route:
  receiver: staff                # fallback receiver
```

```

group_by: ['alertname', 'job']
routes:
  - match_re: { job: "(node|windows)" }
    receiver: infra-email
  - match: { job: kubernetes, severity: ticket }
    receiver: k8s-slack

receivers:
  - name: 'k8s-slack'
    slack_configs:
      - channel: '#alerts'
        text: 'https://example.com/alerts/{{ .GroupLabels.app }}'
  - name: 'infra-email'
    email_configs: [{ to: 'infra@example.com' }]
  - name: 'staff'
    email_configs: [{ to: 'staff@example.com' }]

```

Matcher syntaxes

Syntax	Use case	Example
match	Exact label equality (all pairs)	match: {job: kubernetes, severity: ticket}
match_re	Regex on a label value	match_re: {job: "(node windows)"}
matchers	Array with operators = != =~ !~	['severity="page"', 'env=~"prod staging"']

Nested (sub) routes

```

route:
  receiver: k8s-email
  routes:
    - match: { job: kubernetes }
      receiver: k8s-email
      routes:
        - match: { severity: pager }
          receiver: k8s-pager # overrides parent ONLY for severity=pager

```

A child route only applies if its parent matches first. If the child doesn't match, the alert falls back to the parent's receiver.

continue — deliver to multiple receivers

```

route:
  routes:
    - receiver: alert-logs
      continue: true # keep evaluating LATER routes too
    - match: { job: kubernetes }
      receiver: k8s-email

```

Grouping

```

route:
  group_by: ['team'] # top level grouping
  routes:
    - match: { team: infra }
      group_by: ['region', 'env'] # child route can refine grouping
      receiver: infra-email

```

Reloading

```

sudo systemctl restart alertmanager
# OR
sudo killall -HUP alertmanager
# OR

```

```
curl -X POST http://localhost:9093/-/reload
```

Alertmanager does NOT auto-reload edits — you must trigger one of the methods above.

7.6 Receivers & Notifiers

A **receiver** groups one or more **notifiers** (Slack, email, VictorOps, PagerDuty, webhook...).

```
receivers:
- name: infra-pager
  slack_configs:
    - api_url: https://hooks.slack.com/services/XXXX
      channel: "#pages"
  email_configs:
    - to: "receiver@example.com"
      from: "mail@example.com"
      smarthost: smtp.gmail.com:587
      auth_username: "mail@example.com"
      auth_password: "password"
```

Avoid duplication

Move shared secrets (API keys, SMTP creds) into `global` so every receiver can reference them instead of repeating credentials.

```
global:
  victorops_api_key: "XXX"
receivers:
- name: infra-pager
  victorops_configs: [{ routing_key: some-route }]
```

Templating notification content

Field	Meaning
GroupLabels	Labels used to group the notification
CommonLabels / CommonAnnotations	Values shared by every alert in the group
Status	firing or resolved
Alerts	Array of individual alerts (each with Labels/Annotations)

```
slack_configs:
- api_url: https://hooks.slack.com/xxx
  channel: '#alerts'
  title: '{{ .GroupLabels.severity }} alerts in region {{ .GroupLabels.region }}'
  text: >
    {{ .Alerts | len }} alerts:
    {{ range .Alerts }}{{ .Annotations.description }}{{ "\n" }}{{ end }}
```

7.7 End-to-End Alertmanager Demo

1. Write alert rules (`/etc/prometheus/rules.yaml`):

```
groups:
- name: my-alerts
  interval: 15s
  rules:
    - alert: NodeDown
      expr: up{job="node"} == 0
```

```

for: 0m
labels: { team: infra, env: prod }
annotations: { message: "{{ .Labels.instance }}" is currently down" }
- alert: DatabaseDown
  expr: up{job="db1"} == 0
  for: 0m
  labels: { team: database, env: prod }
  annotations: { message: "{{ .Labels.instance }}" is currently down" }

```

2. **Reference the rule file** and **point Prometheus at Alertmanager** in `prometheus.yml` :

```

rule_files: ["rules.yaml"]
alerting:
  alertmanagers:
    - static_configs: [{ targets: ["localhost:9093"] }]

```

Restart Prometheus, check the Alerts tab.

3. **Configure routing + Slack receiver** in `alertmanager.yml` :

```

route:
  group_by: ['alertname']
  group_wait: 30s
  group_interval: 1m
  repeat_interval: 2m
  receiver: 'web.hook'
  routes:
    - match_re: { job: '(node|db1|db2)' }
      group_by: ['team', 'env']
      receiver: slack
receivers:
  - name: slack
    slack_configs:
      - api_url: 'https://hooks.slack.com/services/...'
        channel: '#alerts'
        title: '{{ .GroupLabels.team }} has alerts in env: {{ .GroupLabels.env }}'
        text: '{{ range .Alerts }}{{ .Annotations.message }}{{ "\n" }}{{ end }}'

```

4. Restart Alertmanager and confirm alerts arrive in Slack, grouped by `team / env` as configured.

Setting	Purpose
<code>group_wait</code>	How long to wait for more alerts before the FIRST notification of a new group
<code>group_interval</code>	How long to wait before sending the next batch for an existing group
<code>repeat_interval</code>	How often to re-send while an alert keeps firing

7.8 Silences

Purpose

Suppress NOTIFICATIONS (not the underlying evaluation) during planned maintenance, deployments, or known transient conditions — so on-call isn't paged for expected noise.

1. Open the **Silences** tab in the Alertmanager UI → New Silence.
2. Set a start time and duration (or explicit end time).
3. Add one or more label **matchers** to scope which alerts are affected, e.g. `job="ec2"`.
4. Add a creator name and comment describing WHY (for future on-call context).
5. Save. The silence appears with status `pending`, `active`, or `expired`.

Status	Meaning
<code>pending</code>	Scheduled to start later
<code>active</code>	Currently suppressing notifications

expired	End time passed, no longer applies
---------	------------------------------------

Important

A silence does NOT stop rule evaluation — the alert still fires and is still visible in the UI. Only the outbound NOTIFICATION is suppressed.

Scope narrowly

A broad matcher like `environment="production"` can hide unrelated, genuinely critical alerts. Prefer the narrowest matcher set that covers exactly your maintenance window.

End of guide — built from Prometheus Certified Associate (PCA) course notes. All commands and PromQL should be validated against your own environment/version before production use.